



El VPS de producción

De un VPS Contabo recién contratado a "Docker operativo y DNS resolviendo", con los comandos exactos. A partir de ahí sigue `11-puesta-en-produccion.md` desde su paso 3. Lo que no se ha podido ejecutar está marcado.

Qué cubre este documento y qué no

`11-puesta-en-produccion.md` describe el procedimiento **genérico**: sirve para cualquier VPS. Este documento es el **concreto** de la máquina que se ha contratado, y sustituye a los pasos 1 y 2 de aquel.

Paso	Dónde está
Contratar, entrar, endurecer, Docker, DNS	Aquí
El <code>.env</code> , los medios, construir y levantar	<code>docs/11</code> , paso 3 en adelante
Copias, restauración, Cloudflare por delante	<code>DESPLIEGUE.md</code>

La máquina: Contabo Cloud VPS 6 — 6 vCPU, 12 GB de RAM, 200 GB SSD, 300 Mbit/s, **x86_64**, Ubuntu 24.04. Es la misma arquitectura sobre la que el proyecto está ensayado en local, así que no hay nada que adaptar.

Aviso honesto sobre el proveedor. Contabo da mucha máquina por poco dinero, y a cambio el rendimiento es **más variable** que en otros proveedores: el vecino de al lado puede quitarte CPU (lo verás como *steal time* en `top`). Para esta carga —marcar una foto cuando alguien abre un pase— es asumible. El soporte también es más lento; si la máquina se cae un domingo, no esperes respuesta hasta el lunes. Conviene saberlo antes, no después.

1. La primera conexión

Contabo entrega la máquina con **acceso de root por contraseña**. Eso es lo primero que hay que quitar, y hasta que lo hagas la máquina está expuesta a fuerza bruta desde el minuto uno.

```
ssh root@<IP-DEL-VPS>
```

No ejecutado aquí: no hay VPS todavía. Los datos de acceso llegan por correo tras el aprovisionamiento, que Contabo puede tardar en completar.

Lo primero, actualizar y poner la máquina en hora:

```
apt update && apt upgrade -y
timedatectl set-timezone Europe/Madrid
```

2. Un usuario que no sea root

```
adduser vistta
usermod -aG sudo vistta
```

Y su clave pública, **desde tu máquina, en otra terminal**:

```
ssh-copy-id vistta@<IP-DEL-VPS>
```

Si no tienes par de claves todavía:

```
ssh-keygen -t ed25519 -C "vistta"
```

Comprueba que entras por clave antes de tocar nada más, en una terminal nueva y **sin cerrar la que ya tienes abierta**:

```
ssh vistta@<IP-DEL-VPS>
sudo -v          # que el sudo funciona
```

3. SSH solo por clave

Con la sesión de arriba **todavía abierta** —si te equivocas y cierras las dos, te quedas fuera de la máquina—:

```
sudo nano /etc/ssh/sshd_config
```

```
PasswordAuthentication no
PermitRootLogin no
KbdInteractiveAuthentication no
```

Y ahora lo que se pasa por alto y hace perder una tarde: en Ubuntu 24.04 el archivo principal incluye `/etc/ssh/sshd_config.d/*.conf`, y las imágenes de VPS suelen dejar ahí un fragmento de cloud-init que vuelve a poner `PasswordAuthentication yes`. **El último que se lee gana**, así que puedes escribir `no` arriba y seguir aceptando contraseñas.

Mira qué queda de verdad, que es lo único que cuenta:

```
grep -r PasswordAuthentication /etc/ssh/sshd_config.d/ 2>/dev/null
sudo sshd -T | grep -E '^(passwordauthentication|permitrootlogin)'
```

Tiene que decir `passwordauthentication no`. Si dice `yes`, corrige el fragmento de `sshd_config.d/` en vez de pelearte con el archivo principal. Después:

```
sudo systemctl restart ssh
```

Y **antes de cerrar la sesión que tienes abierta**, abre otra nueva y entra. Si entra, ya está. Si no, tienes la vieja para arreglarlo.

4. Cortafuegos

```
sudo ufw allow OpenSSH
sudo ufw allow 80
sudo ufw allow 443
sudo ufw --force enable
sudo ufw status verbose
```

Lo que `ufw` **NO** hace, y conviene tener claro: Docker escribe sus propias reglas de iptables y **un puerto publicado con `ports:` se salta `ufw`**. No es un fallo de configuración, es cómo funciona. La consecuencia práctica: **nunca publiques Postgres "protegido por el cortafuegos"**, porque no lo estaría. En `compose.prod.yml` ni `db` ni `api` publican puerto —solo `caddy`, en 80 y 443, que es justo lo que aquí se abre— y esa decisión está comentada en el propio archivo. No la cambies.

5. Docker

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker vistta
```

El grupo no se aplica a la sesión en curso: **sal y vuelve a entrar**, y comprueba que funciona **sin `sudo`** antes de seguir:

```
exit
ssh vistta@<IP-DEL-VPS>
docker run --rm hello-world
```

6. Actualizaciones de seguridad automáticas

```
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades    # responde «Sí»
```

Comprobar que hará algo, sin esperar al primer día:

```
sudo unattended-upgrade --dry-run --debug | tail -20
```

Lo que esto no cubre. Parchea el sistema del VPS, **no las imágenes de Docker**: el Node y las bibliotecas que corren dentro del contenedor vienen de `node:22-bookworm-slim` y solo se actualizan **reconstruyendo la imagen**. Reconstruir de vez en cuando forma parte del mantenimiento, no es opcional.

7. Disco

Los 200 GB son mucho más de lo que esto necesita, porque **los medios no viven aquí**: van a R2. Todo lo que ocupa el despliegue está bajo `/var/lib/docker` del disco raíz.

Qué	Cuánto ocupa
Imagen de la API (<code>vistta-api</code>)	~445 MB
Imagen del panel y viewer (<code>vistta-web</code>)	~85 MB
<code>postgres:16-alpine</code>	~115 MB
<code>pgdata</code> recién creada, con el esquema aplicado	73 MB (medido)
Medios	0 : están en R2
<code>./copias</code>	crece: un volcado por día, 14 días por defecto

O sea: menos de **1 GB** para empezar, sobre 200. Lo único que crece sin techo son las copias y las capas viejas de Docker cada vez que reconstruyes. Cuando el disco baje, esto es lo que lo recupera:

```
df -h /
docker system df
docker system prune -f      # capas e imágenes sin usar; NO toca volúmenes
```

`docker system prune` **no borra volúmenes** sin `--volumes`, y `pgdata` es un volumen. No le añadas ese parámetro sin pensarlo dos veces: es la base de datos.

8. Snapshots de Contabo

El plan incluye **2 snapshots**. Un snapshot es una foto de la máquina entera, y sirve para volver atrás cuando algo sale mal.

La norma: un snapshot **antes de cada cambio de riesgo**. Migrar los medios a R2, actualizar una versión mayor de Postgres, cambiar el sistema operativo. Cuestan un minuto y se hacen desde el panel de Contabo.

Un snapshot NO sustituye a `scripts/backup.sh`. Son cosas distintas y hacen falta las dos:

	Snapshot	<code>backup.sh</code>
Qué guarda	La máquina entera	Solo la base, en formato <code>custom</code> (<code>-Fc</code>)
Cuándo	A mano, antes de algo arriesgado	Cada día a las 4:15, por cron
Dónde vive	En Contabo, con la propia máquina	En el disco, y debe copiarse fuera
Sirve para	Volver atrás de un cambio	Recuperar datos, restaurar tabla a tabla

Un snapshot que vive en el mismo proveedor que la máquina no te salva de perder la cuenta. Y ninguno de los dos guarda los medios: esos están en R2.

No ejecutado: no hay acceso al panel de Contabo. Los nombres exactos de los menús pueden no coincidir; el concepto sí.

9. DNS en dinahosting

Esto es lo que hay hoy, comprobado con `dig` desde fuera:

Registro	Valor actual	Qué es	Qué hacer
<code>A</code> <code>@</code>	<code>82.98.135.43</code> (<code>redirecciones.dinaser.com</code>)	El parking de dinahosting	Cambiar a la IP del VPS
<code>A</code> <code>www</code>	<code>82.98.135.43</code>	El mismo parking	Cambiar a la IP del VPS (pero lee el aviso de abajo)
<code>MX</code>	<code>10 mail.vistta.es</code>	El correo	NO TOCAR
<code>A</code> <code>mail</code>	<code>82.98.134.111</code> (<code>d876.dinaser.com</code>)	El servidor de correo	NO TOCAR
<code>NS</code>	<code>ns.dinahosting.com</code> y tres más	Quién manda en la zona	No tocar

Por qué el correo sobrevive: el `MX` apunta a `mail.vistta.es`, que tiene **su propio registro `A`** en otra IP (`82.98.134.111`, un servidor distinto del parking). Cambiar `@` y `www` no lo roza. **El peligro real es borrar registros en bloque:** si en el panel eliges algo del estilo «eliminar todos los registros A» para limpiar el parking, te llevas por delante el de `mail` y el correo deja de entregarse sin que nada más se rompa —así que tardarás en enterarte—.

Pon el TTL en **300 segundos** mientras montas: si te equivocas, se corrige en cinco minutos en vez de en un día. Súbelo cuando esté estable.

Comprueba ANTES de levantar Caddy

```
dig +short vistta.es          # la IP del VPS
dig +short www.vistta.es      # la IP del VPS
dig +short mail.vistta.es     # 82.98.134.111, SIN CAMBIOS
dig +short vistta.es MX       # 10 mail.vistta.es.
```

Las cuatro tienen que salir bien **antes** de arrancar Caddy. Let's Encrypt limita los intentos fallidos por dominio y por hora: si arrancas con el DNS a medias, te quedas sin poder pedir certificado durante un rato largo, y no por un fallo tuyo del momento sino por los intentos que ya gastaste.

Sobre `www`

El `deploy/Caddyfile` de hoy solo atiende `{${DOMINIO}}`. No hay bloque para `www`, así que si apuntas `www.vistta.es` al VPS, quien la escriba se encontrará un error de TLS: Caddy no tiene certificado para ese nombre y no sabe qué servir. Dos salidas, y hay que elegir una:

- **No crear el registro** `www` apuntando al VPS. Sencillo, pero quien teclee `www` no llega.
- **Añadir un bloque de redirección** al `Caddyfile`, que es lo normal en un sitio público:

```
www.{${DOMINIO}} {
    redir https://{${DOMINIO}}{uri} permanent
}
```

Pendiente de decisión, y por eso no está hecho: tocar el `Caddyfile` es un cambio de configuración del despliegue, no de este documento. Mientras no se decida, **no apuntes** `www` al VPS.

Una cosa que se vio de camino, y que no es de este despliegue

El dominio **no tiene registro SPF** (`dig +short vistta.es TXT` no devuelve nada). Con el correo alojado en dinahosting, eso hace que los mensajes que salgan de `@vistta.es` tengan más papeletas de acabar en spam. No afecta a Vistta ni a este despliegue, pero ya que vas a estar en el panel del DNS, es el momento de mirarlo.

Cuando algo falla

Síntoma	Causa probable	Qué hacer
Sigues entrando por contraseña tras ponerlo <code>no</code>	Un fragmento de <code>sshd_config.d/</code> lo sobrescribe	<code>sudo sshd -T grep passwordauthentication</code>
<code>docker</code> pide <code>sudo</code>	El grupo no se aplica a la sesión en curso	Sal y vuelve a entrar por SSH
Caddy no consigue certificado	El DNS no apunta aquí todavía	<code>dig +short vistta.es</code> ; espera y reinicia Caddy
<code>www</code> da error de TLS	El <code>Caddyfile</code> no atiende <code>www</code>	Ver el apartado de arriba: es una decisión pendiente
El correo deja de llegar	Se borró el <code>A</code> de <code>mail</code> al limpiar el parking	Recrear <code>mail</code> → <code>82.98.134.111</code>
La máquina va a tirones	<i>Steal time</i> del vecino	<code>top</code> , columna <code>st</code> . Es el proveedor, no tu código

Y a partir de aquí

Con Docker operativo y el DNS resolviendo, sigue en `11-puesta-en-produccion.md`, **paso 3**: el `.env`, dónde van los medios, construir y levantar. Con 12 GB, construir en el propio servidor es lo normal: no hace falta traer las imágenes desde fuera.

Qué se ha ensayado y qué no

Comprobado de verdad	El estado del DNS de <code>vistta.es</code> (con <code>dig</code> , desde fuera), las IP del parking y del correo, y lo que ocupan las imágenes y una <code>pgdata</code> recién migrada (73 MB)
Nunca ejecutado	Todo lo que ocurre dentro del VPS : no hay máquina todavía. Ni el endurecimiento del SSH, ni Docker, ni <code>ufw</code> , ni los snapshots de Contabo, ni el cambio del DNS

Los comandos son los estándar de Ubuntu 24.04 y cada apartado dice cómo comprobar que hizo efecto — `sshd -T`, `ufw status`, `hello-world`, `dig`—. Esa comprobación es la que convierte esto en un procedimiento y no en una lista de buenos deseos: **hazla**.